MCP Demystified

Your First Connection

Connect an AI to the real world - and watch it work.

MS 1.1	Cline + Gemini	Windows
Skill Sprint	Free Tools	OS

The Problem

boss@company.com
URGENT: Client escalation today — URGENT

teammate@company.com
Quick question about the Q3 deck — NEEDS REPLY

newsletter@medium.com
10 articles you might like this week — IGNORE

billing@software.com
Your invoice is ready — FYI

recruiter@linkedin.com
Exciting opportunity - fast-growing — IGNORE

Every morning...

☕ Coffee gets cold

🕐 1+ hour lost to triage

🧠 Mental energy drained

😓 Replies pile up

📅 Meetings booked manually

What if an agent handled all of this?

What is MCP?

A standard protocol for AI models to use real-world tools.

BEFORE MCP

Custom code for EVERY tool.

Gmail: bespoke integration.

Notion: another bespoke.

Slack: another bespoke.

Endless glue code 😩

→

WITH MCP

One protocol. Every tool.

Build a server once.

Any model can use it.

Swap tools without rewriting.

This is USB for AI. 🔌

The 3 Pieces of MCP

Host · Client · Server — memorise these three words.

01 MCP Host

Your workspace - where YOU work

The app you interact with. For us: Cline running inside VS Code. The host is where you type, see results, and control everything.

→ Cline (free) in VS Code

02 MCP Client

The translator inside the host

Lives inside the host. Speaks the MCP protocol - sends tool requests to servers, gets results back, passes them to the model. Cline handles this automatically.

→ Built into Cline, you don't write it

03 MCP Server

The gateway to a real tool

Knows how to talk to a specific external service. Gmail MCP Server knows the Gmail API. Calendar MCP Server knows Google Calendar. You run these yourself.

→ Open-source, self-hosted, free

The Full MCP Flow

Real example happening today:

You type: **"Read the file test_mcp.txt on my Desktop"** → Cline → Filesystem MCP Server → reads the actual file → model reports back.

📁 *This same loop is how it will read your Gmail inbox on Wednesday.*

The 100% Free Stack

No credit card. No subscriptions. No catch.

Layer	✅ Free Path (Use This)	💳 Paid Upgrade (Optional)
💻 **IDE**	VS Code + Cline	*Cursor Pro ($20/mo)*
🧠 **LLM**	Gemini free tier — gemini-2.0-flash	*Claude API / GPT-4o*
📧 **Email**	smtplib / self-hosted Gmail MCP server	*SendGrid / Resend*
📅 **Calendar**	Google Calendar API + MCP server	*Cal.com API*
🖥️ **UI**	Streamlit / Python CLI	*Retool / Railway*

Setup in 4 Steps (Windows)

1

VS Code

Already installed? Open it. If not: code.visualstudio.com → download .exe

✓ You can open VS Code and see the editor

2

Node.js

node --version → should show v18+
If missing: nodejs.org → LTS installer → run .msi

✓ Terminal shows v18.x.x or higher

3

Cline Extension

VS Code → Extensions (Ctrl+Shift+X) → search 'Cline' → Install

✓ Cline icon appears in your sidebar

4

Gemini Free API Key

aistudio.google.com → Sign in → Get API Key → Create key → Copy it
In Cline: select Google Gemini, paste key, model = gemini-2.0-flash

✓ Ask Cline 'what model are you?' — it replies

Connect Your First MCP Server

The Filesystem Server — safe, zero-auth, instant proof

Step 1 — Install the server

In VS Code terminal:

```
npx -y
@modelcontextprotocol/server-filesystem
--help
```

Step 2 — Create a test file

On your Desktop, create test_mcp.txt with any text inside.

Step 3 — MCP Config (paste into Cline → MCP Servers):

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [ "-y", "@modelcontextprotocol/server-filesystem",
                "C:\\Users\\YourName\\Desktop" ]
    }
  }
}
```

⚠️ **Windows paths need DOUBLE backslashes in JSON: C:\\Users\\YourName\\Desktop**

The Payoff Moment 🎉

Type this into Cline →

> *"Can you read the file called test_mcp.txt on my Desktop and tell me what's in it?"*

What happens next:

1. Cline sees your request and identifies that 'filesystem' server is relevant
2. It calls the MCP server's `read_file` tool with your Desktop path
3. The server reads the actual bytes off your Windows filesystem
4. The result flows back to the model — which tells you exactly what's in the file

This exact same loop → Wednesday → Gmail threads. Thursday → email triage. Saturday → your full inbox.

Triage Preview — Sprint Build 1

Let's see where this leads before we wrap up.

Create sample_threads.txt and run:

```
Read sample_threads.txt.
Act as an email triage assistant.
Classify each thread as:
URGENT / NEEDS-REPLY / FYI / IGNORE
Give a brief reason for each.
```

Agent output →

URGENT	**Client escalation today** Time-sensitive, from boss
NEEDS REPLY	**Q3 deck question** Colleague waiting on you
FYI	**Invoice ready** No action needed yet
IGNORE	**Newsletter articles** Promotional, low value
IGNORE	**Recruiter opportunity** Unsolicited outreach

Saturday: 20 real Gmail threads → same output from your actual inbox.

🏅 **Sprint Build 1 Proof: 20 real threads auto-sorted into priority buckets — public post**

What You Did Today

✓	01	Understand MCP — a standard protocol for AI tools
✓	02	Know the 3 pieces: Host, Client, Server
✓	03	Cline + Gemini running on your Windows machine
✓	04	First MCP server connected (filesystem)
✓	05	Watched an AI read a real file via the protocol

📸 **Screenshot your triage output → post on LinkedIn → #MyAIChiefOfStaff**

Tomorrow — MS 1.2

Inbox X-Ray

Swap the filesystem server for the Gmail MCP server.
OAuth once → read your real inbox.

Homework tonight: go to console.cloud.google.com
Create a new project →
'MCP Chief of Staff'